Pleated Ribbon Construction: Approximate Sorting Recovers the Locality of a Full Sort

Brian Sam-Bodden
Integrallis Software
Scottsdale, Arizona, USA

Abstract

Probabilistic membership filters let storage systems skip reads for keys that are not present. Ribbon filters use much less memory than Bloom filters at comparable false-positive rates, but their higher construction cost limits deployment. The published homogeneous ribbon algorithm sorts keys by start position, at least approximately; the reference implementation and RocksDB insert them in arrival order instead. I ask how coarse that sort can be while keeping its benefit. A counting-sort partition into cache-sized windows of start positions, with no order inside a window, recovers 98% of the cache-miss reduction of a full sort at about one quarter of its reorder cost. At 100–400 million keys, sequential construction is 1.68× faster than with the best full sort I measured and 2.05–2.24× faster than the reference build. The gain comes from reducing misses in dependent row-reduction chains; for the tested Bloom construction and filter queries, whose misses are independent, software prefetch already recovers most of what batching can.

I prove that banding followed by back-substitution produces the same filter under any insertion order, for a fixed hash seed and table size, whenever the system is consistent, which a homogeneous ribbon system always is; reordered builds matched the arrival-order build in every measured run and had no false negatives. A slot-range parallel implementation takes 11.9 ns/key at 100 million keys with 16 threads. The method also transfers to RocksDB's shipped Standard128 ribbon builder, where filter construction is 1.8× faster at 100 million keys per filter, with no gain for filters that fit in cache, and measured false-positive rates match stock to six digits. In the evaluated compaction configuration, the total CPU reduction is within run-to-run variation. These results apply to the evaluated filters, parameters, and machines; they do not establish a general construction or deployment crossover with Bloom.

1 Introduction

A membership filter is a small in-memory summary of a large key set. Ask it whether a key is present, and it answers "definitely not" or "probably yes." The first answer is the one worth having: a database that keeps a filter in front of each data file skips reading any file whose filter says no. Bloom filters [1] have done this for five decades, at a cost of about 44% more memory than the theoretical minimum for a given error rate. A newer generation (xor, binary fuse, and ribbon filters) narrows that gap, to under 10% for ribbon and under 1% in ribbon's bumped variants [2, 3].

The new filters are static: each is built once over a fixed key set and never updated. That suits write-optimized databases, whose data files are immutable and are rewritten continuously by background compaction. But every rewrite also rebuilds the file's filter, so build speed matters as much as query speed, and building is where the new filters lose. A ribbon filter costs about four times the CPU of

a Bloom filter (RocksDB measures 140 vs. 32 ns per key [4]), so production systems keep Bloom as the default and reserve the space-efficient filters for data they rarely rewrite. The authors of these structures report the same cost. The parallel-BuRR authors write that "[a] large portion of the construction time is spent on parallel sorting" [5], and a recent proposal reports building several times slower than the best existing builders [6].

The published homogeneous-ribbon algorithm already contains a remedy: it sorts the keys by start position, "at least approximately" [7]. The reference implementation and RocksDB do not perform that step; they insert keys in arrival order behind a prefetch pipeline. A full sort costs more than it should have to, so the question of this paper is how coarse the order can be while keeping the sort's benefit, and why. The answer is that it can be very coarse. Keys only need to be grouped into windows of start positions that fit in cache, in any order within a window, because what the order buys is locality for a chain of dependent memory accesses. Out-of-order execution and software prefetch already overlap independent stalls; they cannot overlap a chain in which each access depends on the previous one.

Contributions.

1. A cache-sized partition keeps the locality of a full sort (§3). At scale, homogeneous ribbon banding incurs 5.8 generic hardware cache misses per key in dependent row-reduction chains, which prefetch cannot hide. The ribbon authors' own homogeneous-ribbon algorithm already asks for the keys to be sorted by start position "at least approximately" [7] (and already in its technical report [8]) for a step-count reason, but the reference implementation and RocksDB band in arrival order, and I found no implementation or measurement of the step. I implement it as a cache-sized counting-sort partition and measure it: grouping keys into L2-sized windows of start positions, with no ordering inside a window, recovers 98.3–98.4% of the miss reduction of a full start-position sort (SGAUSS [9], BuRR [3]), at 5.5–6.1 ns/key, where BuRR's own ips2ra sorter costs 23.9–28.1. End to end, construction is 1.68× faster than with the best full sort and 2.05–2.24× faster than the reference prefetch-pipelined build, which does not sort. The result is insensitive to window size: totals stay within 13% from 2^{13} to 2^{20} slots.
2. The solved filter does not depend on insertion order (§5). For any consistent system, banding and back-substitution give the same filter in every order (Proposition 1, proved in Appendix A), even though the redundant keys that are dropped can differ; a homogeneous ribbon system is always consistent. This extends the ribbon authors' observation that the set of filled slots is order-invariant. My implementation still compares each result's FNV-1a fingerprint with the arrival-order build and checks for false negatives; the fingerprints matched in all 54 runs of the main sequential experiment and in every parallel run. A slot-range parallel variant that defers boundary keys (at most 0.23% of them) can therefore be checked against the sequential result with a checksum: banding is 4.25× faster at 8 threads and 7.56× at 16 against its own single-thread run (3.5× and 6.3× against sequential pleated banding). The total is 11.9 ns/key at 100M keys.
3. The technique transfers to RocksDB's shipped Standard128 ribbon builder (§7). One counting pass makes filter construction 1.8× faster at 100M keys/filter and yields 25× fewer banding cache misses at the same instruction count. RocksDB's tests pass, measured false-positive rates match stock to six digits, and in a separate check, all 443 filters I compared were byte-identical to stock. In the compaction configuration I evaluated, which favors filters, the filter-build gain persisted, but the reduction of about 9% in total compaction CPU was within run-to-run variation. I also give what is, to my knowledge, the first evaluation of the construction prefetch that RocksDB ships behind a "TODO: verify/validate" comment (§6): it is worth 1.5–1.7× on unsorted banding, and pleating recovers most of its benefit.
4. Two negative results mark where reordering does not help (§8). Batched probing did not win: miss-dominated point probes are limited by the number of outstanding misses a core can sustain, and software prefetch already recovers most of what that limit allows. Cross-key SIMD batching loses to per-key probing even with every address L1-resident. Bulk Bloom construction gained little beyond what a prefetch loop gives, for the same reason: its misses are independent, so they can be overlapped. Radix-partitioned construction in the style of Schmidt et al. [10] topped out at 1.45–1.71× over per-key insertion; a prefetch loop of the kind RocksDB ships without a published evaluation recovered most of that with no

transient memory; and my read-free register-aggregated apply, the design the experiment was built around, was 1.5× slower than partitioned read-modify-write, because partitioning had already turned the target misses into L2 hits.

The primary x86 measurements use an i9-14900HX (8 performance and 16 efficiency cores; AVX2, no AVX-512). Comparative numbers come from the field's reference benchmark suites, run unmodified at pinned versions: fastfilter_cpp (whose ribbon implementations are the reference code of ribbon's creator, Peter Dillinger) and BuRR. Throughout, "reference" means this reference *implementation*, which bands keys in arrival order behind a prefetch pipeline; it omits the approximate sort that the published homogeneous-ribbon algorithm lists [7]. The only benchmark code I wrote for the homogeneous-kernel study is a thin driver that permutes the input order around the unmodified kernel; its parallel variant additionally uses a boundary-checked copy of the reference banding loop. I also report ARM timing replications and a separate Graviton4 PMU follow-up (§4). For each experiment, the hypothesis and the rule for deciding it were written down and committed before data collection, and results that went against a prediction are reported with the others. Raw outputs and analyses are in the repository.

2 Background

Ribbon banding. A ribbon filter solves a banded GF(2) system. Each key hashes to a start column s and a w-bit coefficient row, and insertion XOR-reduces that row against the rows already stored until an empty slot absorbs it. Dietzfelbinger and Walzer showed that sorting rows by start position makes banded elimination linear (SGAUSS). For standard ribbon, the ribbon paper then made the sort optional: rows can be inserted on the fly, in arbitrary order, and "[t]here is no need to pre-sort the keys" [2]. RocksDB's builder and the reference implementation work this way. BuRR brought the sort back (ips2ra, by bucket) because bumping needs it. Homogeneous ribbon, the variant I target, never fails construction and stores no per-key result rows. For this variant the published algorithm is not order-free. Its construction listing in the full version of the BuRR paper [8] has the step "sort S approximately by $s(x)$", and the accompanying proof notes that it is enough if keys are "sorted into buckets of b consecutive starting positions each and buckets handled from left to right." The unified journal treatment [7] keeps the step as "sort S by $s(x)$", annotated "at least approximately." The stated reason is a bound on the row operations spent on redundant insertions, not memory locality. That bound does not limit construction in my runs: at 100M keys, banding executes 97.4 instructions per key in arrival order, 96.2 after a full sort, and 97.8 after partitioning. Order changes where banding touches memory but not how much work it does. The same papers observe that the set of filled slots does not depend on insertion order. Coarse bucketing by start position is therefore the ribbon authors' idea. I found no implementation of it in print or in code, no cache-miss account of why it matters, and no measurement of how much of a full sort's benefit it keeps; the reference implementation I benchmark and RocksDB both band in arrival order with a prefetch pipeline. I use *pleating* as a name for the cache-sized instance of that approximate sort.

What the filter stores and how a query works. A ribbon filter is an $m \times r$ bit matrix Z. Hashing maps each key k to three values: a start position $s(k)$, a w-bit coefficient row $c(k)$ whose first bit is always 1, and r result bits $b(k)$. The filter represents k by satisfying one linear equation over GF(2): $c(k) \cdot Z = b(k)$, with $c(k)$ occupying columns $[s(k), s(k)+w)$. A query recomputes s, c, and b and answers "present" if and only if $c \cdot Z = b$. A key that was never inserted passes that test by chance with probability 2^{-r}, which is the false-positive rate; at $r = 7$, that is just under 0.8%.

Why banding is local. Construction solves the linear system incrementally: each insertion XOR-reduces its row against stored rows until an empty slot absorbs it. Because a key's coefficients span only w consecutive columns, all of the work for that key happens in a run of slots beginning at $s(k)$, and the expected run is short [2]. The techniques in this paper rely on that locality: grouping keys by start region confines banding's memory traffic to one region at a time, and a thread that owns a range of slots can process every key whose run stays inside it.

The homogeneous variant. Homogeneous ribbon stores no per-key result rows during banding and never fails: a row that reduces to zero is dropped, trading a small false-positive elevation for guaranteed completion. My configuration throughout is $w = 64$, $r = 7$, and $m \approx 1.09n$ slots, which gives 7.63 bits per key at a measured false-positive rate of 0.82%.

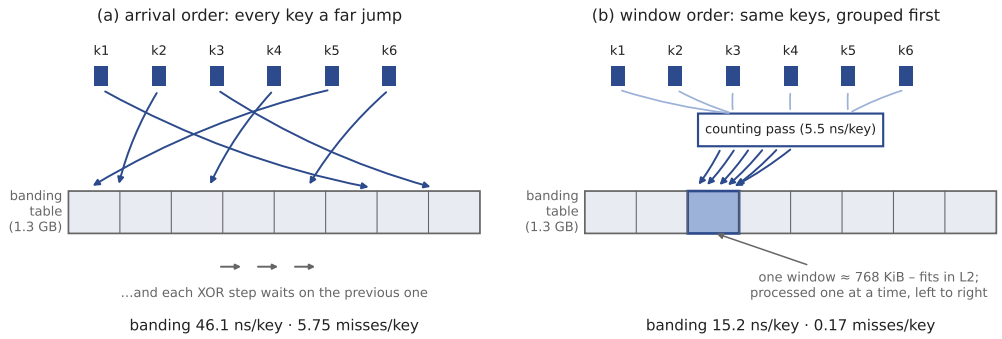


Figure 1: Effect of insertion order on banding. (a) In arrival order, every key starts its elimination at a random position in a table far larger than the L2 cache, and the steps within each insertion form a dependent chain no prefetcher can run ahead of. (b) A counting pass assigns keys to table windows, and a scatter pass groups them by window; banding then works inside one cache-resident window at a time. Binary fuse filters apply the same remedy to their counter scatter via a segment sort (§2). Numbers: 100M keys, means of 3 runs.

Binary fuse. Binary fuse construction partially sorts keys by segment, explicitly to reduce cache misses [11]; I measured that fix directly (plain xor: 9.45 generic hardware cache misses/key; binary fuse: 1.08). This is the closest analogue to my method outside the ribbon family: a single linear pass into coarse position buckets, motivated by cache behavior, and not a full sort. Binary fuse ships the pass as part of its published construction; the ribbon implementations I measured do not have one.

3 Partition-instead-of-sort for ribbon

3.1 The target, measured

I first measured where ribbon construction spends its time, using the reference benchmark suite unmodified (three repetitions). At 100 million keys, homogeneous ribbon construction costs 50.6 ns per key, and the hardware counters show why: the processor completes only 0.64 instructions per cycle and takes 5.81 generic hardware cache misses per key, so it spends most of its time waiting on memory. Each key starts its elimination at an effectively random position in a 1.3 GB table, and every reduction step needs the result of the previous XOR before it can compute which slot it touches next. A prefetcher cannot run ahead of such a chain. Bloom construction is the opposite case: each key's memory accesses are independent, and the hardware overlaps them.

3.2 Design

I only permute the input. One counting pass buckets keys by $\lfloor s/2^{16} \rfloor$ (windows of 2^{16} slots, roughly 768 KiB of banding state, under half an L2 cache), recomputing the hash on the fly instead of materializing pairs; a second pass writes the keys out in window order. The unmodified reference kernel, including its prefetch pipeline, then consumes the permuted array. Everything downstream (banding, back-substitution, queries) is reference code, so the speedup is attributable to reordering alone. Figure 1 shows the change in access pattern.

Cost in memory. The pass is not in place. It allocates one additional array of n 64-bit keys, 8 bytes per key, which is transient and is freed after banding; the window counters are negligible, and the parallel variant copies each thread's keys once more. Measured peak resident memory agrees: at 100M keys the pleated build peaks 8.0 bytes per key above the reference build, against 23.0 for the ips2ra sort in the same driver (Appendix D, which also states what each reported time includes).

3.3 Results

Table 1 gives the results at 100M keys, and Table 2 gives the totals at 10M, 100M, and 400M. The partition pass costs 5.5–6.1 ns/key against 23.9–28.1 for ips2ra in my driver (17.3 inside BuRR's

Table 1: Homogeneous ribbon construction, 100M keys, ns/key (mean ± sample stdev over 3 reps; parallel rows 2 reps; raw artifacts in repository). Banding perf counters scoped to the banding phase. The miss/key column is Linux's generic hardware cache-miss event on x86, not a direct count of DRAM transactions.

strategy	reorder	banding	total	cache-misses/key
unsorted, no prefetch	—	76.9 ± 4.3	80.8 ± 4.2	5.81
unsorted + prefetch (ref.)	—	46.1 ± 1.4	49.6 ± 1.4	5.75
full sort (std::sort)	76.8 ± 7.1	15.5 ± 2.2	96.1 ± 8.9	0.074
full sort (LSD radix)	23.4 ± 1.9	14.7 ± 0.8	41.7 ± 1.2	0.076
full sort (ips2ra)	23.9 ± 0.1	13.4 ± 0.1	40.7 ± 0.3	0.075
partitioned (this work)	$\mathbf{5.5 \pm 0.1}$	15.2 ± 0.1	$\mathbf{24.2 \pm 0.2}$	0.170
parallel, 8T (this work)	6.4	4.3 ± 0.4	14.6 ± 2.1	—
parallel, 16T (this work)	5.7	2.4 ± 0.4	$\mathbf{11.9 \pm 0.8}$	—

Table 2: Construction by key count on the i9-14900HX, for the reference build (ref.), the best full sort (sort), and the partitioned build (part.): total ns/key (means of 3 reps), how many times faster the partitioned build is than each of the others, and banding cache misses per key. Best full sort: the fastest of the three sorts at that size (ips2ra at 10M and 100M, LSD radix at 400M).

keys	ref. ns	sort ns	part. ns	vs. ref.	vs. sort	ref. miss	sort miss	part. miss
10M	42.3	39.1	23.9	1.77×	1.64×	3.78	0.074	0.164
100M	49.6	40.7	24.2	2.05×	1.68×	5.75	0.075	0.170
400M	55.3	41.5	24.7	2.24×	1.68×	6.13	0.075	0.170

own benchmark). Banding after partitioning is within 13% of banding after a full sort (15.2 vs. 13.4 ns/key); that is, the partition recovers 98.3–98.4% of the sort's miss reduction (5.75 → 0.17 vs. 0.075 misses/key). End to end, construction takes 24.2 ns/key against 49.6 for the reference and 40.7 for the best full sort (Figure 2), which is 1.68× faster than the best full sort, as it is at 400M keys. BuRR is included only as a mechanistic anchor: it uses a different, bumped structure ($r = 8$, about 1% space overhead) than my homogeneous configuration (7.63 bits/key at 0.82% FPR, about 9.5% overhead), so I do not compare end-to-end totals with it. A window sweep from 2^{13} to 2^{20} slots moves totals only between 23.8 and 26.9 ns/key, and the choice of 2^{16}, fixed before the sweep, was conservative: 2^{14} is slightly faster (Appendix E, Table 8).

The remaining gap to Bloom. In the same harness and at close false-positive rates, ribbon construction goes from 4.4× the cost of blocked Bloom to 2.2×: pleating roughly halves the gap but does not close it (Appendix E, Table 7). The comparisons against a production Bloom builder and at equal thread counts are in §7 and §5.

4 Portability across architectures

I repeated the construction measurements on two ARM cores: an Ampere Altra (Neoverse-N1, NEON) and a Google Axion (Neoverse-V2, SVE), each on a cloud host with at most eight cores. Table 3 gives wall-clock results at 100M keys. Pleating is faster on all three machines, by a wider margin on the ARM cores: 2.05× over the reference build on x86, 2.37× on N1, and 2.72× on V2. This is consistent with the mechanism: the less effectively a core hides the dependent-chain misses, the more removing them helps. Reference banding is costlier on both ARM cores (46 ns/key on x86 against 114 on N1 and 86 on V2), although the speedup does not follow that cost in order: V2 gains most. Parallel construction scales near-linearly on the eight-core V2: banding speeds up 1.94/3.78/7.07× at 2/4/8 threads, for a 17.3 ns/key total (one run per thread count, so no variance estimate).

The output checks passed on all three machines. At each of the nine (size, seed) points, the 64-bit FNV-1a solution fingerprint matched across the Axion, x86, and N1 builds; every strategy also matched the sequential build's fingerprint. These are empirical hash comparisons, not direct byte-wise comparisons, and the order-related part is what Proposition 1 predicts. Identity across machines

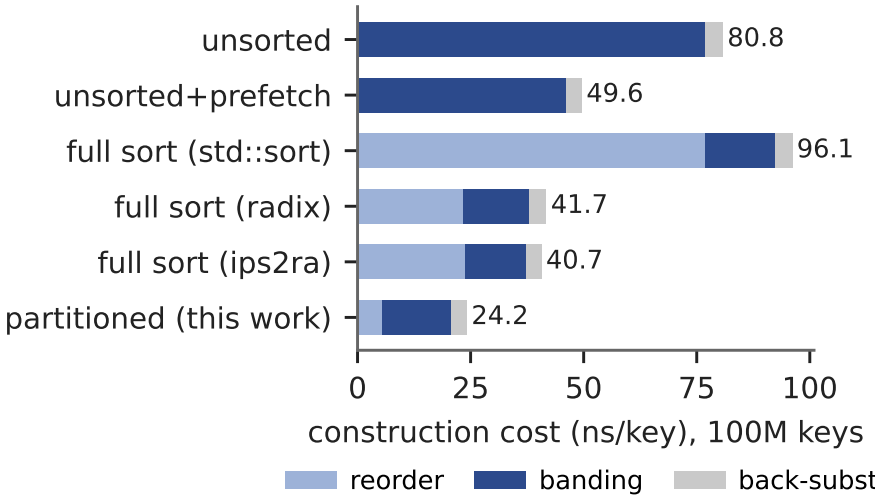


Figure 2: Construction cost by phase, 100M keys. The partition pass replaces the sort at $4\times$ lower reorder cost, while banding stays within 13% of banding after a full sort.

Table 3: Construction on three machines, 100M keys, wall-clock ns/key (mean $\pm$ sample stdev over 3 reps). These original cross-architecture runs report timing and output checks; the separate Graviton4 PMU follow-up is described in the text.

	x86 Raptor Lake	N1 (NEON)	V2 (SVE)
reference	49.6 ± 1.4	122.8 ± 0.3	93.7 ± 0.5
best full sort	40.7 ± 0.3	64.6 ± 0.3	44.8 ± 0.3
partitioned	$\mathbf{24.2 \pm 0.2}$	$\mathbf{51.7 \pm 0.3}$	$\mathbf{34.4 \pm 0.0}$
pleating vs reference	2.05×	2.37×	2.72×

additionally reflects that the hashing and the GF(2) arithmetic are deterministic. There were zero false negatives on every run.

A separate Graviton4 run with counter access measures the mechanism on Neoverse-V2: the partition recovers 96.8% of the full sort's L2 read-miss reduction (Appendix E).

5 Order-independence and parallel construction

Proposition 1 (Order-independence) *Let $E = \{(r_k, b_k)\}$ be a consistent system of equations over $\mathrm{GF}(2)$ in m unknowns, for one fixed hash seed and table size, each r_k carrying a leading 1 at its start position. Then banding (reduce each row against the stored rows, store it at the slot of its leading position, and drop it if it reduces to zero) followed by back-substitution in which every free slot j is assigned a value $g(j)$ that depends only on j (not on the table's contents or on the insertion history) yields the same solution Z under every insertion order of E.*

The rows need not be linearly independent. A homogeneous ribbon system has $b_k = 0$ for every key and is therefore always consistent, so a homogeneous ribbon filter does not depend on insertion order at all. In the kernels I use, a standard ribbon build for a given seed fails only when a row reduces to zero with a nonzero right-hand side, so it succeeds exactly when its system is consistent, and that too is a property of E, not of the order: if E is inconsistent, every order meets such a row. The proposition does not cover schemes that bump or give up on keys depending on their position in the order, as BuRR does.

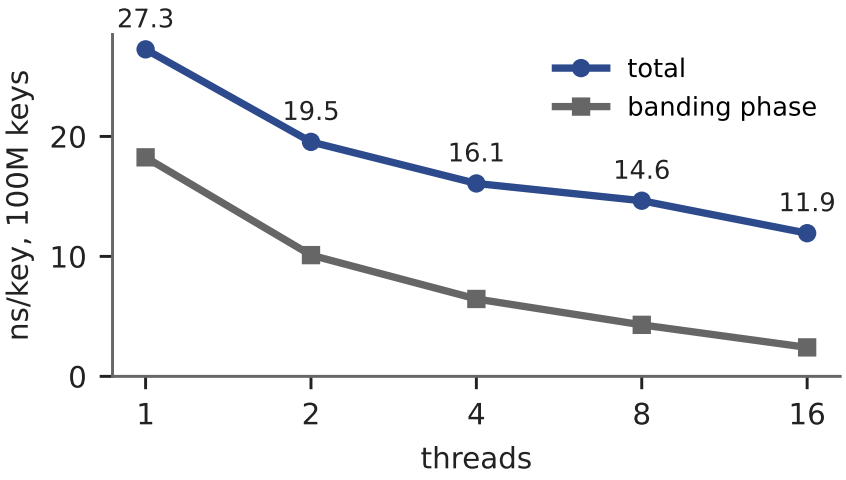


Figure 3: Slot-range parallel construction, 100M keys; every point has a matching 64-bit solution fingerprint against the sequential build.

The first step of the proof is known: the ribbon papers state that the set of filled slots is invariant under insertion order [2, 7]. I extend that observation to the solved filter; the proof, which is elementary linear algebra, is in Appendix A. The measurements agree: across every homogeneous sequential and parallel run (sizes up to 400 million keys, six reordering strategies, up to 16 threads) the finished filter's 64-bit FNV-1a fingerprint matched the arrival-order build, and all 443 RocksDB standard ribbon filters I compared were byte-identical to stock (§7). I keep the per-run fingerprint comparison in the implementation as a guard against bugs, not because the result depends on it. My parallel version assigns disjoint window ranges (split at the same key count) to threads; keys starting within 2^{14} slots of a range boundary switch to a sequential tail (0.015–0.226% of keys), following the boundary handling of parallel BuRR. BuRR makes shards independent by bumping every key whose equation spans a shard boundary to the next layer [7, 5]; a ribbon without bumping has no next layer, so I defer those keys within the same system instead. Safety does not rest on that margin: a thread never writes at or beyond the upper boundary of its range, and a reduction that would reach it is handed to the sequential tail instead (Appendix D). Banding scales 1.81/2.83/4.25/7.56× at 2/4/8/16 threads against its own single-thread run (Figure 3), or 3.5/6.3× at 8/16 threads against sequential pleated banding (15.2 ns/key), because the parallel path carries about 20% overhead for deferring and copying keys. The counting pass (5.7 ns/key) and back-substitution (3.8 ns/key at 16 threads) remain sequential and together account for most of the 11.9 ns/key total. Both can be parallelized, the counting sort directly and back-substitution as in parallel BuRR; I leave both to future work.

Against Bloom at equal thread counts. Bloom insertion is order-free and parallelizes with little effort, so a 16-thread ribbon build should be compared with a 16-thread Bloom build. On a 16-core machine, pleated ribbon costs 4.0× the best Bloom build at one thread and 10.1× at 16. The gap widens because only banding is parallel in my build (Appendix C, Table 6).

6 Construction prefetch, evaluated

RocksDB ships a software-pipelined construction prefetch behind a comment reading "TODO: verify/validate", and the reference harness enables the same pipeline above 1500 slots. My no-prefetch control isolates its effect: 1.5–1.7× on unsorted banding (76.9 → 46.1 ns/key at 100M). The prefetch removes the miss that locates a key's start but cannot shorten the dependent reduction chain, which is why partitioning dominates here while a similar prefetch strategy wins for Bloom construction (§8).

Table 4: Pleating RocksDB's shipped Standard128 ribbon (`filter_bench`, 10 bits/key, i9-14900HX; means of 3). The measured false-positive rate matches stock to six digits at every size. Banding generic hardware cache misses/key from counters scoped to the banding call.

keys/filter	stock ns/key	pleated ns/key	speedup
10^5 (cache-resident)	57.6	59.4	0.97×
10^6	55.9	54.4	1.03×
10^7	88.7	54.7	1.62×
10^8	**97.4**	**54.8**	**1.78**×
banding misses/key @10^8	11.5	0.47	—

7 Pleating a production engine: RocksDB

The ribbon measurements so far use a reference-kernel harness on homogeneous $w = 64$ ribbon. The filter that RocksDB ships is different: `Standard128RibbonBitsBuilder`, a *standard* ribbon at $w = 128$ [4]. I therefore tested whether pleating transfers to it in RocksDB's own build path. The patch is one counting pass over the collected key hashes, inserted immediately before RocksDB's banding call (`ResetAndFindSeedToSolve`); everything downstream is unmodified engine code. As written, the patch is not in place: it holds a second copy of the collected 64-bit hashes and a 32-bit window index per key, 12 bytes (96 bits) per key of transient memory, where RocksDB reports about 230 bits per key of temporary memory for ribbon construction [4]. In `filter_bench` at 100M keys per filter the process peaks at 4.6 GiB with the pass and 3.6 GiB without (Appendix D). An integration meant for upstream would need to reduce that and to bypass the pass for small filters. Proposition 1 applies to this builder one seed at a time: whether a seed succeeds does not depend on order, so the same seed is selected, and the solution for that seed is the same (§5). This depends on where the pass is placed. RocksDB derives the starting seed and the filter length from the first collected hash, and my reorder runs after both are fixed; a reorder placed earlier could change them. I also tested the output directly. RocksDB's own filter tests pass unchanged, and the measured false-positive rate matches stock to six digits at every reported size. That gate alone does not establish bit identity, so I checked identity directly in a separate experiment: the same binary dumps every finished filter, and I compared stock and pleated output byte by byte at 100K, 1M, 10M, and 100M keys per filter, with a second stock run as a determinism control. All 443 filters were byte-identical, and the control matched. The comparison covers the stored metadata, including the hash seed, so the same seed was selected in every case, and byte-identical filters answer every query identically. This holds for the filters built in that experiment; it is not a proof for all inputs. Every measurement first confirms that the binary contains the patch (object recompiled, binary newer than source, marker string present), because an early build failure had left a stale stock binary in place.

Construction microbenchmark (`filter_bench`, the engine's own tool; 10 bits/key). Table 4 reports total filter-construction cost, while Figure 4 plots the banding phase alone. At 100M keys per filter, pleating builds RocksDB's ribbon $1.8\times$ faster overall. The pleated build cost is nearly flat with filter size (54.4–55.5 ns/key from 1M to 100M keys), while the stock build cost rises from 55.9 to 97.4 ns/key as the banding table outgrows the cache. Below the cache crossover (a filter of 10^5 keys fits in cache), pleating is a few percent *slower* because the counting pass has a cost and there is no locality to recover, as the mechanism predicts.

Mechanism. The cause is the same dependent-miss chain, which I confirm with hardware counters scoped to RocksDB's banding call. At 100M keys, pleating reduces banding cache misses from 11.5 to 0.47 per key (a $25\times$ reduction) while the instruction count is unchanged (243 per key in both builds). The work is the same; the working set now fits in cache. The time speedup is smaller than for $w = 64$ because the standard $w = 128$ kernel does more computation per key, and pleating largely subsumes RocksDB's construction prefetch (Appendix E).

End-to-end during compaction. In `db_bench`, banding inside compaction drops from 77.6 ± 1.5 to 44.2 ± 6.1 ns/key, and filter banding is about 23% of compaction CPU in the configuration tested. Total compaction CPU falls by about 9%, which is within run-to-run variation, so I report the whole-compaction effect as directional only (Appendix E).

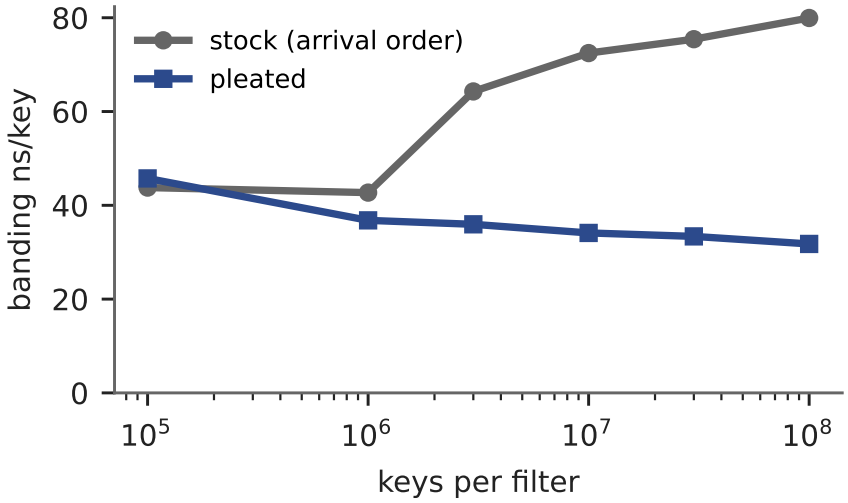


Figure 4: RocksDB Standard128 ribbon banding-phase cost vs. filter size. Pleated banding cost does not rise (cache-resident per window); stock banding degrades as the table outgrows cache. Measured false-positive rates match stock to six digits at every point.

Against RocksDB's own Bloom. At a matched false-positive rate in the same harness, the pleated builder costs 2.4× RocksDB's production Bloom builder at 100M keys per filter, down from 4.5× for stock; below the cache crossover, pleating does not help. That experiment, on a second x86 machine, is in Appendix B (Table 5).

8 Where reordering does not help

This section reports two optimizations that failed. Each was tested under favorable conditions, with its protocol fixed before any run: the batched probe was timed with every address already in cache, and the read-free builder was the design its experiment was built around. Both lost. An optimization that cannot win under conditions chosen to favor it points to a limit of the hardware and not of the setup.

8.1 Point probes are MLP-bound

I compared probe strategies over identical split-block Bloom filters (Parquet geometry), 32 KiB up to 512 MiB (the instrument was amended once after a first run; the data reported are from the amended run). My vertical, lane-structured batch probe auto-vectorizes from plain Rust to packed SIMD, yet it loses. It loses to per-key probing at every size: to plain scalar probing up to 4 MiB, and to scalar probing with software prefetch at RAM sizes. Even with every address forced L1-hot it reaches 555 Mops/s against 665 for scalar probing at 32 KiB, because a split-block check is already SIMD within a single key, and an AVX2 gather cannot beat eight independent scalar loads that the out-of-order window overlaps at no cost. At RAM scale, the identical instruction stream runs 5–6× faster when L1-hot, so probing is bound by memory-level parallelism. Software prefetch recovers part of that and then plateaus (148 Mops/s at 64 MiB against 89 without it), which is what a limit on the misses one core can keep outstanding would produce. No compute bottleneck remains for a layout change to remove.

8.2 Blocked-Bloom construction is prefetch-hidable

I measured five bulk builders at 1M to 429M keys, each required to produce bit-identical output (OR is commutative, so any deviation is a bug). Per-key insertion does degrade at scale (one random read-modify-write miss per key, 11.5 ns/key at 100M, IPC 0.48), so there is a cost to recover. Most attempts to recover it failed. Radix-partitioned construction after Schmidt et al. [10] reached only

1.45–1.71×. A software-pipelined prefetch insert, the shape RocksDB ships, reached 6.1–8.9 ns/key with no transient memory and beat partitioning up to 100M keys. The variant this experiment was designed around is a read-free apply that partitions pre-merged (block, mask) pairs, OR-aggregates each block in registers, and issues one store per block without reading filter memory. It was 1.5× slower than partitioned read-modify-write, and non-temporal stores made no difference. The explanation is that partitioning had already turned the expensive misses into L2 hits, so the extra grouping pass spent time to save cheap loads. Bloom construction misses are independent, and the memory system already hides independent misses.

9 Related work

Sorted banded elimination: Dietzfelbinger and Walzer [9], BuRR [3], and parallel BuRR [5]; the unified journal treatment of standard, homogeneous, and bumped ribbon is Dietzfelbinger et al. [7]. Approximate sorting of homogeneous-ribbon input into buckets of consecutive start positions is specified there and in the BuRR technical report [8], with a row-operation bound as its rationale (§2). Arbitrary-order banding and production engineering: ribbon [2], RocksDB [4]. Cache-conscious filter design: buffered Bloom filters [12], radix-partitioned filters [10], binary-fuse segment sort [11], and xor-filter buffered scatter [13]; sharding the structure itself for construction locality in hypergraph-based static functions [14, 15] and sort-based cache-oblivious peeling [16]; the generic two-phase partition-then-accumulate pattern appears as propagation blocking [17] with write-combining from Wassenberg and Sanders [18]. SIMD probe-side layout: Polychroniou and Ross [19], Lang et al. [20], split-block Bloom [21]. The ordering idea is therefore not mine. What was open is how coarse the order may be, what it costs, and why it works. BuRR sorts fully (ips2ra) to enable threshold-based bumping and tighter space, and bumping needs that order, so I make no claim that a coarse partition transfers to BuRR. For homogeneous ribbon, the authors specify an approximate sort, but the implementations I found do not perform one. I build it as a counting-pass partition sized to the cache, show that it captures 98% of the sort's locality at 4× less reorder cost, explain the gain by dependent-miss chains, and apply the same pass to RocksDB's standard ribbon builder, whose design forgoes sorting. Unlike sharding, the partition only permutes the input and leaves the stored structure unchanged, which makes the transformation and its parallelization checkable against the arrival-order build.

10 Limitations and threats to validity

I characterize the miss mechanism primarily with x86 hardware counters; a separate AWS Graviton4 run also measures the Neoverse-V2 L2 read-miss event (§4). Since the ARM event differs from the x86 generic cache-miss event, I compare reductions within each machine and do not pool absolute counts. The original ARM timing replications confirm wall-clock speedups and matching output fingerprints. I accelerate the *homogeneous* ribbon (about 9.5% space overhead at $r = 7$), whereas BuRR has under 1% overhead; BuRR is a mechanistic anchor, not a like-for-like baseline. The sort baselines use BuRR's sequential ips2ra but not its parallel form. The partition pass needs transient memory (8 bytes per key in the standalone driver, 12 in the RocksDB patch); peak memory and concurrent builds were measured on one machine only, with separate processes standing in for an engine's concurrent compactions. Only banding is parallelized, and the scaling ratios are against the parallel path's own single-thread run. The order-independence proposition covers systems that are solved exactly for one fixed seed and table size; it says nothing about schemes that bump keys by their position in the order. The fingerprint comparisons test the implementations against it, not the proposition itself. The RocksDB construction gain is measured on the production $w = 128$ kernel; byte identity with stock is shown for the 443 filters I compared, not for all inputs. The whole-compaction result comes from one configuration that favors filters: the measured reduction is directional, falls within run-to-run variation, and depends on SST sizing.

11 Conclusion

In the measured configurations, a counting-sort partition recovers 98% of the full sort's cache-miss reduction for homogeneous ribbon construction at about a quarter of the reorder cost. Reordering does not change the output: for a fixed hash seed and table size, the solved filter is the same in every

insertion order for any consistent system, which covers every homogeneous build and every successful standard build, and the measured fingerprints agree. The method also speeds up RocksDB's shipped $w = 128$ ribbon builder by $1.8\times$ at 100M keys per filter. RocksDB's tests pass, measured false-positive rates match stock to six digits, and all 443 stock and pleated filters compared in a separate experiment were byte-identical. The measured whole-compaction change is directional and configuration-dependent. Against RocksDB's own Bloom builder at a matched false-positive rate, pleated ribbon still costs $2.4\times$ as much to build at 100M keys per filter ($4.5\times$ for stock) on one thread. In a separate equal-thread comparison of homogeneous ribbon with my blocked-Bloom prototype, not matched on space or false-positive rate, the gap grows from $4.0\times$ at one thread to $10.1\times$ at 16, because only banding is parallel in my build. These results hold for the evaluated filters and machines; they narrow ribbon's single-thread build-cost gap to Bloom but do not close it.

Artifacts. All protocols, amendments, raw measurements, and integrity tooling are in the accompanying repository (`https://github.com/integrallis/pleated-ribbon-construction`). `reproduce.sh` re-derives every analysis from committed raw measurements, checks the Rust tests and artifact integrity, and regenerates figures. It does not rerun the measurements themselves; the registered protocols and runners for those are in the repository. A production implementation of pleated construction (tested byte-for-byte against the reference kernel) is released as the `pleat` crate at `https://github.com/integrallis/pleat`.

Acknowledgements. I thank Stefan Walzer for helpful comments.

References

[1] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *CACM*, 13(7), 1970.

[2] Peter C. Dillinger and Stefan Walzer. Ribbon filter: Practically smaller than bloom and xor. arXiv:2103.02515, 2021.

[3] Peter C. Dillinger, Lorenz Hübschle-Schneider, Peter Sanders, and Stefan Walzer. Fast succinct retrieval and approximate membership using ribbon. In *SEA*, 2022.

[4] Peter C. Dillinger. Ribbon filter. RocksDB blog, rocksdb.org/blog/2021/12/29/ribbon-filter.html, 2021.

[5] Christopher Becht, Hans-Peter Lehmann, and Peter Sanders. Brief announcement: Parallel construction of bumped ribbon retrieval. arXiv:2411.12365, 2024.

[6] Antoine Limasset. ZOR filters. arXiv:2602.03525, 2026.

[7] Martin Dietzfelbinger, Peter C. Dillinger, Lorenz Hübschle, Peter Sanders, and Stefan Walzer. Ribbon: Fast succinct static retrieval and approximate membership. *J. ACM*, 73(1), 2026. Article 7. doi:10.1145/3785417.

[8] Peter C. Dillinger, Lorenz Hübschle-Schneider, Peter Sanders, and Stefan Walzer. Fast succinct retrieval and approximate membership using ribbon. arXiv:2109.01892 (full version of the SEA 2022 paper), 2021.

[9] Martin Dietzfelbinger and Stefan Walzer. Efficient Gauss elimination for near-quadratic matrices with one short random block per row, with applications. In *ESA*, 2019.

[10] Tobias Schmidt, Maximilian Bandle, and Jana Giceva. A four-dimensional analysis of partitioned approximate filters. *PVLDB*, 14(11), 2021.

[11] Thomas Mueller Graf and Daniel Lemire. Binary fuse filters: Fast and smaller than xor filters. *ACM JEA*, 27, 2022.

[12] Mustafa Canim, George A. Mihaila, Bishwaranjan Bhattacharjee, Christian A. Lang, and Kenneth A. Ross. Buffered Bloom filters on solid state storage. In *ADMS@VLDB*, 2010.

[13] Thomas Mueller Graf and Daniel Lemire. Xor filters: Faster and smaller than bloom and cuckoo filters. *ACM JEA*, 25, 2020.

[14] Marco Genuzio, Giuseppe Ottaviano, and Sebastiano Vigna. Fast scalable construction of (minimal perfect hash) functions. In *SEA*, 2016.

[15] Sebastiano Vigna. ε-cost sharding: Scaling hypergraph-based static functions and filters to trillions of keys. arXiv:2503.18397, 2025.

[16] Djamal Belazzougui, Paolo Boldi, Giuseppe Ottaviano, Rossano Venturini, and Sebastiano Vigna. Cache-oblivious peeling of random hypergraphs. arXiv:1312.0526, 2013.

[17] Scott Beamer, Krste Asanović, and David Patterson. Reducing PageRank communication via propagation blocking. In *IPDPS*, 2017.

[18] Jan Wassenberg and Peter Sanders. Engineering a multi-core radix sort. In *Euro-Par*, 2011.

[19] Orestis Polychroniou and Kenneth A. Ross. Vectorized bloom filters for advanced simd processors. In *DaMoN*, 2014.

[20] Harald Lang, Thomas Neumann, Alfons Kemper, and Peter Boncz. Performance-optimal filtering: Bloom overtakes cuckoo at high throughput. *PVLDB*, 12(5), 2019.

[21] Jim Apple. Split block bloom filters. arXiv:2101.01719; Apache Parquet format specification, 2021.

A Proof of Proposition 1

Setting. The unknown is $Z \in \mathrm{GF}(2)^{m\times r}$. Each equation is a pair (r_k, b_k) of a nonzero row $r_k \in \mathrm{GF}(2)^m$ and a result $b_k \in \mathrm{GF}(2)^r$, and asks for $r_k Z = b_k$. Write $\mathrm{lead}(v)$ for the first nonzero position of a nonzero row v. Banding keeps a table in which slot i is empty or stores a pair (ρ_i, β_i) with $\mathrm{lead}(\rho_i) = i$. To insert (ρ, β): while $\rho \neq 0$ and slot $\mathrm{lead}(\rho)$ is occupied, add that slot's pair to (ρ, β), which moves $\mathrm{lead}(\rho)$ strictly to the right; then store the pair at slot $\mathrm{lead}(\rho)$ if $\rho \neq 0$, and drop it otherwise. Stored pairs are never modified. Back-substitution visits the slots from m down to 1, setting $Z_j = g(j)$ for an empty slot j and $Z_i = \beta_i + \sum_{j>i} \rho_{i,j} Z_j$ for an occupied slot i, where Z_j is row j of Z. Let $V = \mathrm{span}\{r_k\}$ and $P(V) = \{\mathrm{lead}(v) : v \in V,\ v \neq 0\}$. Vectors with distinct leading positions are linearly independent, and every subspace has a basis in echelon form, so $|P(V)| = \dim V$.

Proof. Fix any insertion order and consider the table after the last insertion.

(i) The occupied slots are exactly $P(V)$. Every stored pair is a sum of equations of E, so every stored row lies in V. The stored rows have distinct leading positions, so they are linearly independent. Each r_k equals the remainder it was reduced to, which is zero or a stored row, plus the stored rows that were added to it; hence the stored rows span V and form a basis of it. The occupied slots are therefore $\dim V$ distinct elements of $P(V)$, which has $\dim V$ elements. This step is the invariance of the set of filled slots that the ribbon papers state [2, 7].

(ii) The stored system has the same solutions as E. A solution of E satisfies every sum of equations of E, so it satisfies the stored system. Conversely, an equation (r_k, b_k) that was stored after reduction is the sum of its stored remainder and other stored pairs, so the stored system implies it. An equation that was dropped satisfies $r_k = \sum_{i\in I} \rho_i$ and $b_k = \beta' + \sum_{i\in I} \beta_i$ for a set I of occupied slots, where β' is the result it was left with. Since E is consistent, it has a solution Z^0, which also satisfies the stored system, so $b_k = r_k Z^0 = \sum_{i\in I} \rho_i Z^0 = \sum_{i\in I} \beta_i$ and $\beta' = 0$. The dropped equation is thus a sum of stored equations, and the stored system implies it as well.

(iii) The output is determined by E. By the recursion that defines it, the output Z of back-substitution satisfies every stored equation and has $Z_j = g(j)$ for every empty slot j; by (i) and (ii) it is a solution of E with $Z_j = g(j)$ for all $j \notin P(V)$. There is only one such solution: two of them agree on all rows outside $P(V)$, both satisfy the stored system, and the stored equation at slot i determines row i from rows $j > i$, so they agree on every row by induction from m downwards. The solution set of E, the set $P(V)$, and g do not depend on the insertion order, so neither does Z. □

Remarks. Which equations are dropped does depend on the order, as the ribbon authors note [7]; the solution does not. If E is inconsistent, every order meets a row that reduces to zero with a

Table 5: RocksDB `filter_bench` build cost (ns/key, means of 3) for the production Bloom filter and Standard128 ribbon, stock and pleated, at 10 bits/key setting (AMD EPYC-Milan, 8 dedicated vCPUs, KVM). Last column: measured false-positive rate %, Bloom / ribbon.

keys/filter	Bloom	stock	pleated	stock/Bloom	pleated/Bloom	FP%
100,000	15.9	78.9	82.2	5.0×	5.2×	0.97 / 0.95
1,000,000	20.2	94.2	83.4	4.7×	4.1×	0.97 / 0.95
10,000,000	22.6	164.6	92.9	7.3×	4.1×	0.96 / 0.96
100,000,000	37.1	167.8	90.3	4.5×	2.4×	0.97 / 0.95

nonzero result: otherwise every dropped equation would be a sum of stored equations, as in (ii), and the stored system, being in echelon form, has a solution, which would then solve E. So whether a standard ribbon build succeeds for a given seed is also independent of the order. The reference implementation meets the condition on g: it fills free solution rows with a fixed multiplicative hash of the slot index, as the ribbon authors describe [7]; zero is the special case $g \equiv 0$.

Tests of the implementations. The proposition is about the abstract procedure; whether a given kernel implements that procedure is a separate question, which I tested. On the unmodified reference kernel, homogeneous filters sized so that up to half of the keys are dropped as redundant came out identical under 25 shuffled orders, and standard builds either succeeded identically in every order or failed in every order. As a check on the statement itself, an exhaustive enumeration of all small systems (up to seven slots and five or six equations, every insertion order) found no violation.

B Bloom and ribbon in RocksDB at matched false-positive rate

Table 4 compares ribbon with ribbon. To compare both with Bloom, I ran a further experiment: `filter_bench` building RocksDB's production Bloom filter (`format_version` 5, which ships its own construction prefetch) and Standard128 ribbon, stock and pleated, from the same key stream at the same bits-per-key setting, three repetitions, arms interleaved. RocksDB sizes its ribbon to the Bloom filter's false-positive rate, so the arms are matched by construction: measured rates were 0.96–0.97% for Bloom and 0.95–0.96% for ribbon, with ribbon storing 27–30% fewer bits per key. This run used a different machine from Table 4 (AMD EPYC-Milan, 8 dedicated vCPUs under KVM), so its absolute numbers are not comparable with that table; all three arms ran there in one session. Table 5 gives the result. At 100M keys per filter, the pleated builder costs 2.4× the Bloom builder, down from 4.5× for stock; at 10M it is 4.1×, down from 7.3×. Below the cache crossover pleating does not help (5.2× against 5.0× at 100K). I had written down two predictions in advance. The first, that pleated ribbon would remain more than 1.5× Bloom at 10M keys and above, holds. The second, that it would come within 1.25× of Bloom at 100M, fails, as I expected. At one thread and matched accuracy, pleating therefore narrows ribbon's build-cost gap to Bloom and leaves a factor of 2.4 at the largest size. Part of the narrowing at 100M is Bloom's own cost rising with filter size on this machine (15.9 to 37.1 ns/key). The stock-to-pleated speedup replicates on this second x86 machine: 1.77× at 10M and 1.86× at 100M.

C Bloom and ribbon at equal thread counts

A 16-thread ribbon build should be compared with a 16-thread Bloom build, and Bloom insertion is order-free, so it parallelizes with little effort. I ran that comparison on one machine in one session: an AWS c7a.4xlarge (AMD EPYC 9R14, 16 physical cores), 100M keys. The Bloom side is my blocked-Bloom prototype with three parallel builders (block-range partitioning, atomic ORs on a shared filter, and per-range scanning), each checked for a matching output fingerprint against per-key insertion; at each thread count I take the fastest, so the baseline is the strongest I could build. The ribbon side is the unmodified driver of Table 1. Table 6 gives the result. At one thread, ribbon costs 4.0× the best Bloom build; at 16 threads, it costs 10.1× (22.2 against 2.20 ns/key). The gap widens with threads because my build parallelizes only banding: on this machine the sequential counting pass and back-substitution alone cost about 20 ns/key, and the ribbon total improves 1.8× from 1 to 16 threads, where Bloom improves 4.6×. Both predictions I had written down in advance held: ribbon stays above 1.5× Bloom at every thread count, and it does not come within 1.25× at

Table 6: Equal-thread construction cost at 100M keys (AWS c7a.4xlarge, AMD EPYC 9R14, 16 physical cores). Bloom: the fastest of three parallel blocked-Bloom builders at that thread count (10 bits/key; Criterion mean). Ribbon: pleated homogeneous ribbon total (mean $\pm$ sample stdev of 3; sequential build at one thread).

threads	best Bloom builder	Bloom ns/key	ribbon ns/key	ribbon/Bloom
1	par_scan_t1	10.01	40.30 ± 0.05	4.0×
2	par_scan_t2	8.14	33.89 ± 0.22	4.2×
4	par_range_t4	5.47	27.09 ± 0.15	5.0×
8	par_range_t8	3.30	23.83 ± 0.03	7.2×
16	par_atomic_t16	2.20	22.15 ± 0.07	10.1×

16 threads. This comparison is not matched on space or false-positive rate (Bloom at 10 bits/key and 1.27% measured false positives, ribbon at 7.63 bits/key and 0.83%), and the prototype's sequential build is 22% slower than the reference suite's blocked Bloom on this machine, inside the 25% tolerance I had set for using its numbers. Parallel banding alone is therefore not enough: closing the equal-thread gap would also require parallelizing the counting pass and back-substitution.

D Timing scope, memory, and parallel boundary safety

What the timings include. In the standalone driver, the reorder time includes allocating the permuted array and both passes of the partition, with the hash recomputed in each. Banding and back-substitution times include hashing. Key generation and the allocation of the banding table are outside the timers for every strategy. In RocksDB, `filter_bench`'s build time covers adding the keys and finishing the filter, which includes hashing, the partition pass, any seed retries, and allocation.

Measured memory and concurrent builds. A further experiment measured peak resident memory and concurrent builds on one machine (AWS c7a.4xlarge, AMD EPYC 9R14, 16 physical cores), at 100M keys, three repetitions each. In the standalone driver the whole process peaks at 1.6 GiB for the reference build and 2.4 GiB for the pleated build, a difference of 8.0 bytes per key, as the code predicts. The ips2ra sort in the same driver adds 23.0 bytes per key, because it sorts pairs of start position and key, and the 16-thread parallel build adds 15.1 bytes per key. These peaks include the input keys and the verification pass, so only the differences are attributable to reordering. In RocksDB's `filter_bench` at 100M keys per filter the process peaks at 3.6 GiB for stock and 4.6 GiB with the pass; that tool varies the keys per filter and holds its finished filters, so I report the peaks and not a per-key figure. Compaction usually builds several filters at once, so the same experiment ran 1, 4, and 8 sequential builders at the same time, each pinned to its own core. The pleated build was 2.02×, 2.16×, and 2.20× faster per process than the reference build: the advantage does not shrink when builders compete for memory bandwidth and the shared cache.

Boundary safety. The 2^{14}-slot margin by which keys near a range boundary are deferred is a heuristic, not the safety argument. Safety rests on a hard cap: a thread loads and stores only slots below the upper boundary of its range, and a reduction that would reach that boundary stops without writing and hands its key to the sequential tail. Reductions only move toward higher slots, and every key a thread processes starts inside its own range, so no thread touches a slot below its range either. With the default margin, no reduction reached the cap in any run that recorded the count. To exercise the fallback, I reran the parallel build at 2 and 16 threads with the default margin and with it reduced to 1024, 64, 8, and 0 slots. With margins of 8 and 0, up to 138 reductions in a run reached the cap and were handed to the tail; in all 30 runs (ten settings, three repetitions) the solution fingerprint matched the sequential build and there were no false negatives. Because the tail re-inserts deferred keys into the same system, Proposition 1 applies.

E Additional measurements

Same-harness comparison with blocked Bloom. Table 7 compares the ribbon builds with the reference suite's blocked Bloom in the same harness, on the same machine, at the same key count,

Table 7: Construction cost of blocked Bloom and homogeneous ribbon at 100M keys, single thread, i9-14900HX, same harness, joined from existing raw measurements (no new runs); ns/key is mean $\pm$ sample stdev over 3 reps. Last column: cost relative to the Bloom build.

build	harness	ns/key	FPR%	bits/key	vs. Bloom
blocked Bloom, per-key	fastfilter_cpp	11.21 ± 1.08	0.94	10.67	1.0×
homogeneous ribbon, reference	fastfilter_cpp kernel	49.58 ± 1.43	0.82	7.63	4.4×
homogeneous ribbon, pleated	fastfilter_cpp kernel	24.17 ± 0.21	0.82	7.63	2.2×

Table 8: Partitioned homogeneous ribbon construction by window size, 100M keys, i9-14900HX, ns/key (means of 2 reps) and banding cache misses per key.

window slots	reorder	banding	total	misses/key
2^{13}	6.6	14.3	24.3	0.146
2^{14}	6.1	14.4	23.8	0.149
2^{15}	5.8	14.8	24.1	0.147
2^{16}	5.7	16.0	25.5	0.171
2^{17}	5.3	16.0	24.9	0.182
2^{18}	5.3	16.9	25.6	0.207
2^{19}	5.1	17.6	26.2	0.255
2^{20}	5.2	18.2	26.9	0.306

on one thread, and at close false-positive rates (0.94% and 0.82%), with the ribbon using 29% fewer bits per key. Ribbon construction goes from 4.4× the cost of Bloom to 2.2×. Pleating roughly halves the gap but does not close it. A per-key blocked Bloom is not the fastest Bloom build: my own prefetch-pipelined builder reaches 7.35 ns/key, but it builds a weaker filter (1.27% false positives at 10 bits/key, measured at 1M keys) in a different harness, so I do not tabulate it. The comparison against a production Bloom builder with construction prefetch, at a matched false-positive rate, is summarized in §7 and given in Appendix B (Table 5). The equal-thread comparison with parallel Bloom builds is in Appendix C (Table 6).

Window size. Table 8 gives the sweep over window sizes summarized in §3. Smaller windows cost slightly more in the partition pass and less in banding; the total moves by 13% over a factor of 128 in window size. These are separate runs from Table 1, with two repetitions each, so the 2^{16} row differs from that table's.

Counter measurement on ARM. A separate AWS Graviton4 run (Neoverse-V2, eight vCPUs) enabled user-space PMU access and measured one Neoverse-V2 event scoped to the banding call (`l2d_cache_lmiss_rd`). The median L2 read misses per key were 1.770 for the reference, 0.058 for partitioned construction, and 0.001 for LSD radix sort; the partition recovers 96.8% of the full sort's miss reduction. This is a mechanism check on a second Neoverse-V2 host, separate from the Google Axion timing results in §4. The event is microarchitecture-specific, and its absolute counts are not directly comparable with the x86 event.

Compaction in `db_bench` (`fillrandom` 30M keys, then full compaction, 8-byte values and 256 MB target SSTs, so filters are a meaningful share of compaction; three repetitions). Filter construction runs during background compaction, so it is not on the write path, and write throughput is unchanged. The filter-build speedup persists inside compaction: banding drops from 77.6 ± 1.5 to 44.2 ± 6.1 ns/key (means of 3); the isolated, uninstrumented build shows a larger ratio, which the statistics instrumentation compresses here. Filter banding is about 23% of compaction CPU in this configuration (1.83 s of 7.9 s). The whole-compaction result is weaker: total compaction CPU falls by about 9% ($7.9 \pm 0.8 \rightarrow 7.2 \pm 0.4$ s), but that difference is of the same order as the run-to-run variance of compaction, which is nondeterministic, so I report it as directional only. The effect also grows with per-SST filter size and vanishes below the cache crossover, so its magnitude depends on SST sizing, and this configuration is at the end of that range that favors filters.

Differences from the $w = 64$ kernel. Two things differ from the $w = 64$ case, and both follow from the mechanism. First, standard $w = 128$ spends about 243 instructions per key against about

98 for homogeneous $w = 64$ (128-bit $\mathrm{GF}(2)$ reductions plus a per-seed rehash), so a larger share of each key's time is computation that no layout change can remove. This is why the *time* speedup is $1.8\times$ here against $2.05\times$ for $w = 64$, even though the miss reduction is comparable. Second, RocksDB's shipped construction prefetch (§6) attacks the same start-locating miss; pleating largely subsumes it: at 20M keys per filter (two runs), disabling the prefetch slows the pleated build by about 7% against about 29% for stock.